CENTRO PAULA SOUZA – FATEC BEBEDOURO
Faculdade de Tecnologia Jorge Caram Sabbag
Curso Superior de Tecnologia em Big Data no Agronegócio

Disciplina: Algoritmos Avançados (IAL213) **Semestre/Ano:** 2º Semestre / 2026
Docente: Prof. Me. Douglas Samuel Gonçalves **Ambiente:** Laboratório de Programação
Aluno(a): _____ **Data:** _____

Lista Prática de Exercícios: Aplicações de Grafos em Python

Fixação de Implementação, Estruturas de Dados e Tomada de Decisão no Agronegócio

Orientações Gerais:

- **Premissa Operacional:** Vocês já dispõem do script com a classe `Grafo` e os métodos de travessia e caminho mínimo (`bfs`, `dfs` e `dijkstra`) desenvolvidos em aula.
- **Objetivo Central:** O propósito desta lista **não** é avaliar complexidade assintótica nem reescrever algoritmos teóricos do zero. O objetivo é colocar o Python em prática: manipular dicionários, desempacotar tuplas, iterar sobre vizinhanças, carregar arquivos reais e utilizar o grafo como ferramenta de análise de dados.
- **Entrega:** Submeta um arquivo Python individual (ex.: `exercicios_grafos_seu_nome.py`) contendo as funções solicitadas e seus respectivos testes.

Bloco 1: Aquecimento & Manipulação Elementar de Estruturas

Exercício 1: Modelagem Manual da Fazenda

Uma propriedade rural produtora de cana e grãos na região de Bebedouro possui 4 talhões produtivos (`T1`, `T2`, `T3`, `T4`) e 2 silos intermediários de transbordo (`Silo_Norte` e `Silo_Sul`). A malha rodoviária interna é não-direcionada e possui as seguintes distâncias em quilômetros:

- `T1` conecta-se a `Silo_Norte` (12 km);
- `T2` conecta-se a `Silo_Norte` (8 km) e a `Silo_Sul` (15 km);
- `T3` conecta-se a `Silo_Sul` (10 km);
- `T4` conecta-se a `Silo_Sul` (6 km);
- `Silo_Norte` conecta-se a `Silo_Sul` (18 km).

O que implementar:

a) Instancie um objeto da classe `Grafo(direcionado=False)` e insira todas as conexões acima utilizando o método `adicionar_aresta(u, v, peso)`.

b) Escreva um laço `for` em Python que percorra cada localidade cadastrada e imprima na tela o nome do local acompanhado da lista dos seus vizinhos diretos.

Exercício 2: Inspetor de Gargalos Logísticos (Auditando Graus)

Na logística de escoamento de safras, a localidade que concentra o maior número de conexões rodoviárias atua como o entroncamento crítico (*Hub*) da malha, representando o principal potencial de gargalo em dias de colheita intensa.
O que implementar: Escreva uma função com a assinatura:

```
def encontrar_maior_hub(grafo) -> tuple[str, int]:
```

A função deve iterar sobre todos os vértices da malha, consultar a quantidade de vizinhos de cada um (utilizando `grafo.grau(v)` ou o tamanho de `grafo.obter_vizinhos(v)`) e retornar uma tupla contendo o nome da localidade com maior grau e o respectivo número de conexões.
Teste esperado com o Exercício 1: Retornar `('Silo_Sul', 4)`.

Exercício 3: Carregador Automático a partir de Tuplas

Em aplicações reais de engenharia de dados, conexões e distâncias dificilmente são inseridas uma a uma manualmente; elas costumam chegar estruturadas como coleções de tuplas provenientes de bancos relacionais ou APIs.
O que implementar: Crie uma função genérica:

```
def construir_grafo_de_tuplas(arestas: list[tuple], direcionado: bool = False) ->
                                        Grafo:
```

A função deve instanciar internamente um novo `Grafo`, percorrer a lista de tuplas desempacotando origem, destino e peso (`for u, v, peso in arestas:`), alimentar a estrutura e retornar o objeto pronto.
Dados para validação:

```
dados_safra = [
    ("Sede", "Pivo_1", 4.5),
    ("Sede", "Pivo_2", 7.0),
    ("Pivo_1", "Pivo_2", 3.2),
    ("Pivo_2", "Silo_Graos", 11.0)
]
```

Bloco 2: Consultas, Filtros e Propriedades Estruturais

Exercício 4: Auditoria de Sensores de Campo (Isolados e Pontas)

Em redes de monitoramento IoT com dezenas de sensores de umidade de solo e estações meteorológicas, nós com grau zero indicam perda completa de comunicação (falha de hardware), enquanto nós com grau 1 representam extremidades de linha (*folhas*).
O que implementar: Escreva a função:

```
def auditar_sensores(grafo) -> dict[str, list[str]]:
```

A função deve inspecionar a rede e retornar um dicionário com duas listas categorizadas:

- `"isolados"`: Sensores que possuem exatamente 0 conexões ativas;
- `"pontas"`: Sensores que possuem exatamente 1 conexão ativa.

Cenário de Teste: Crie uma rede com sensores `['S1', 'S2', 'S3', 'S4', 'S5']`, onde há arestas entre `S1--S2` e `S2--S3`, deixando `S4` e `S5` sem conexões. A função deve retornar: `{"isolados": ['S4', 'S5'], "pontas": ['S1', 'S3']}`.

Exercício 5: Estrada Direta e Trecho Mais Longo

Um operador logístico em um ponto de partida `origem` precisa consultar rapidamente a viabilidade de tráfego direto antes de despachar caminhões canavieiros.
O que implementar: Implemente duas funções de suporte:

1. `tem_estrada_direta(grafo, u: str, v: str) -> bool`: Retorna `True` se houver ligação direta entre `u` e `v`, ou `False` caso contrário.
2. `estrada_mais_longa_de(grafo, u: str) -> tuple[str, float]`: Examina todos os vizinhos imediatos do ponto `u` e retorna o nome do vizinho mais distante e a quilometragem correspondente.

Exercício 6: Construção Artesanal da Lista de Adjacência

Para consolidar como uma lista de adjacência opera internamente na memória sem depender da classe pronta, implementaremos o mapeamento utilizando apenas tipos nativos do Python (`dict` e `list`).
O que implementar: Escreva uma função pura:

```
def montar_lista_adjacencia(lista_arestas: list[tuple]) -> dict[str, list[str]]:
```

A função recebe uma lista de conexões não-ponderadas e não-direcionadas no formato `[("A", "B"), ("B", "C"), ("A", "C")]` e constrói artesanalmente o dicionário de listas de adjacência.
Saída esperada: `{"A": ["B", "C"], "B": ["A", "C"], "C": ["B", "A"]}`.

Bloco 3: Aplicações de Travessias (BFS e DFS)

Exercício 7: Rastreamento de Sinal por Raio de Saltos (BFS)

Em uma rede mesh de telemetria no campo baseada no protocolo LoRaWAN, cada repetidor retransmite pacotes de rádio para nós vizinhos. Para economizar bateria, os pacotes possuem um limite máximo de saltos (*hops*) a partir do *Gateway* central.
O que implementar: Utilizando o método `grafo.bfs(inicio)` já implementado no script (que calcula as distâncias em saltos a partir da raiz), desenvolva a função:

```
def dispositivos_ao_alcance(grafo, gateway_id: str, max_saltos: int) -> list[str]:
```

A função deve executar a busca em largura e retornar uma lista contendo apenas os dispositivos cuja distância em saltos seja menor ou igual a `max_saltos`.
Cenário de Teste: Em uma topologia linear `GW -- R1 -- R2 -- S1 -- S2`, uma chamada com `max_saltos = 2` a partir de `GW` deve retornar `['GW', 'R1', 'R2']`.

Exercício 8: Verificação de Acessibilidade em Período de Chuvas (DFS)

Durante períodos de chuvas intensas, estradas de terra secundárias podem ser interditadas. Um agrônomo precisa verificar preventivamente se **existe qualquer caminho transitável** entre um talhão isolado e a oficina mecânica central antes de rebocar um trator avariado.
O que implementar: Crie a função:

```
def existe_caminho(grafo, origem: str, destino: str) -> bool:
```

A função deve executar a travessia (utilizando BFS ou DFS a partir de `origem`) e verificar se o `destino` foi descoberto durante a exploração, retornando `True` se houver conectividade ou `False` se as localidades estiverem em componentes desconexas.

Bloco 4: Caminhos Mínimos Ponderados & Integração com Arquivos

Exercício 9: Roteamento de Menor Frete entre Usinas (Dijkstra)

Um grande produtor colheu sua safra no talhão `Fazenda_Esperanca`. A cooperativa possui acordos comerciais com três unidades industriais de processamento: `Usina_Norte`, `Usina_Sul` e `Usina_Vale`. O caminhão deve ser despachado para a usina que demandar a menor quilometragem total rodada.
O que implementar: Crie a função:

```
def escolher_melhor_destino(grafo, origem: str, lista_destinos: list[str]) ->
                                    tuple:
```

A função deve:

1. Para cada destino presente em `lista_destinos`, executar o método `grafo.caminho_minimo_dijkstra dest)` presente no script da disciplina;
2. Comparar as distâncias acumuladas e identificar a usina com menor custo de deslocamento;
3. Retornar uma tupla contendo: `(nome_usina_vencedora, distancia_km, rota_completa)`.

Exercício 10 (Desafio Integrador): Carga de Dados CSV e Custo de Combustível

O departamento de TI e logística da usina gerou um arquivo de texto estruturado chamado `rotas_agro.csv` contendo trechos rodoviários regionais no seguinte formato tabular:

```
origem;destino;distancia
Monte_Azul;Bebedouro;32.0
Bebedouro;Pitangueiras;38.0
Pitangueiras;Sertaozinho;36.0
Sertaozinho;Ribeirao_Preto;21.0
Monte_Azul;Barretos;65.0
Barretos;Ribeirao_Preto;120.0
```

O que implementar: Desenvolva um programa completo em Python que execute as seguintes etapas integradas:

1. Abra o arquivo `rotas_agro.csv` utilizando o gerenciador de contexto (`with open(...)`) e faça a leitura linha a linha em streaming;
2. Descarte a primeira linha de cabeçalho, utilize o método `.split(";")` para extrair os campos, converta a quilometragem para `float` e popule um `Grafo(direcionado=False)`;
3. Execute o algoritmo de Dijkstra para calcular a rota mínima de `Monte_Azul` até `Ribeirao_Preto`;
4. Supondo que uma carreta canavieira possui consumo médio de combustível de **2,5 km/litro** e que o óleo diesel custa **R$ 6,00 por litro**, calcule e exiba no console:
   - A rota ótima sequencial (ex.: `Monte_Azul -> Bebedouro -> ...`);

- A distância total percorrida em quilômetros;
- O consumo total estimado de combustível em litros;
- O custo financeiro total estimado com combustível em Reais (R$).

Bom Trabalho e Bons Estudos!